```
* ./spark-shell --master local[2]
21/09/01 00:03:04 WARN NativeCodeLoader: Unable to load native-hadoop library for your pla
tform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel
).
Spark context Web UI available at http://192.168.0.97:4040
Spark context available as 'sc' (master = local[2], app id = local-1630434799314).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.1.2
      /_/

Using Scala version 2.12.10 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_121)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Figure 2: Spark shell

commands from the user and submits them to the master. Spark shell supports Scala commands by default.

Start the shell with the following command on a terminal. The command is found under *SPARK_HOME/bin:*

```
spark-shell –-master local[2]
```

This command not only starts Spark shell, but also starts the cluster on the local machine. Since this cluster consists of only one node, obviously the local machine is the master. The number 2 within the brackets results in two worker threads.

It is clear from Figure 2 that the shell has the Spark context, which is accessible as *sc*.

Type the following on the Scala prompt to see the list of available commands:

```
scala> :help
```

Setting up Spark cluster

We have just created a Spark cluster on a single machine. In reality, the cluster consists of several machines. To set up such a cluster, install JDK 8+ and extract the downloaded Spark distribution on all the machines of the cluster.

On one of the machines, run the following command to start the master:

```
SPARK_HOME/sbin/start-master.sh
```

Once the master starts running, find out the master URL from the log file located under the folder *SPARK_HOME/logs*. For example, on the experimental setup, the following is printed as the master URL in the logs:

```
spark://localhost:7077
```

The next step is to set up the workers. Open the terminal on each of the worker nodes, move to the folder *SPARK_HOME/conf* and find the file named *spark-defaults.conf.template*. Make a copy of this file and save it as *spark-defaults.conf*. Open *spark-defaults.conf* and uncomment the line indicating *spark.master*. Edit the URL and other configuration properties, if required.

Once the configuration is ready, start the worker by running the following command:

```
SPARK_HOME/sbin/start-worker.sh
<master-url>
```

For example, on the experimental machine, the worker is started with the following command:

```
SPARK_HOME/sbin/start-worker.sh
spark://localhost:7077
```

Repeat this process on all the workers, to bring up the cluster.

Alternatively, edit the *SPARK_HOME/conf/workers.sh* file on the master node, add the IP addresses of all the worker nodes, and run the command:

```
SPARK_HOME/sbin/start-workers.sh
```

…to bring up the cluster in one shot.

Either way, once the cluster is ready, use the following command to run the Spark shell on a machine that is connected to the cluster:

```
SPARK_HOME/bin/start-shell –master
<master-url>
```

Job submission

The Spark shell can be used to submit jobs to the cluster. For example, the following command submits a job that computes the sum of the specified numbers:

```
scala>  sc.parallelize(Array(1, 2, 3,
4, 5), 2).reduce((a,b)=>a+b)
```

This command refers to an array of five elements as the source data, and submits a reduction function for the computation. Also, the command suggests the data be divided into two partitions.

Obviously, an array of five elements does not represent Big Data. Often, the Big Data is present in local files, HDFS, Cassandra, Hbase, AWS S3, etc, in the production environments. We can point the Spark shell to such data sources.

For example, the following command loads the data file from the local disk and prints the word count:

```
scala> sc.textFile(“data.txt”).
flatMap(line=>line.split(“ “)).count
```

Both the above jobs are small in size. When the job is complex, a separate Spark application can be written in Scala, R, Java, or Python. Such applications are submitted to the